

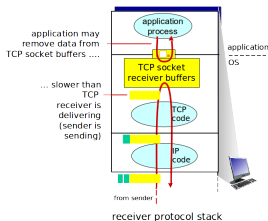
Redes de Datos II

Nivel de Transporte – TCP

Matías Robles

LINTI-UNLP

8 de septiembre de 2026



Contenidos

- 1 TCP - Características
- 2 TCP - Estructura
- 3 Opciones TCP
- 4 Performance TCP
- 5 TCP Selective Acknowledge

Propiedades

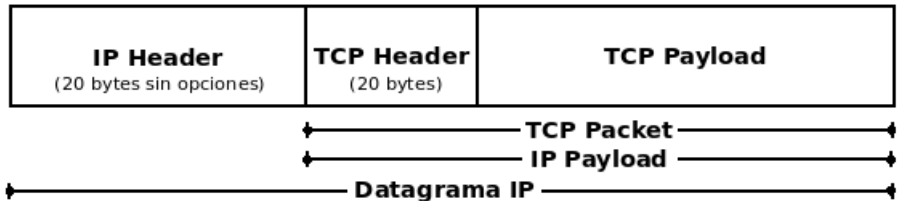
- RFC 793 Std 7, Marzo 1981. Hecho obsoleta por RFC 9293 Std 7, Agosto 2022
- Al igual que UDP, brinda servicios a las aplicaciones, pero le agrega:
 - Orientado a conexión
 - Confiable
 - Ordenado
 - Control de Flujo
 - Control de Congestión
 - Byte stream

TCP - Segmento

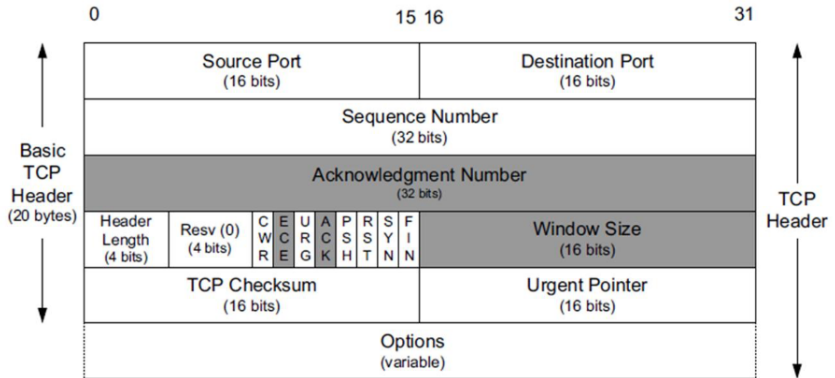
- Aplicaciones envían streams de bytes.
- TCP convierte ese stream en un conjunto de mensajes que IP puede transportar: *packetization*.
- Pueden ser de distinto tamaño y podrían ser combinados (*repacketization*)
- TCP decide el tamaño de cada chunk (pedazo) para que no tenga que ser fragmentado.
- Cada chunk recibe el nombre de *segmento*.
- Cada segmento contiene información de su posición en el stream.
- Cada segmento contiene el proceso origen en el nodo emisor y el proceso destino en el nodo destino.

TCP Encapsulación

- Segmento TCP se encapsula en un paquete IP.
- En la cabecera IP, el campo *Protocol* tiene el valor 6.



TCP - Segmento



TCP Header - Campos

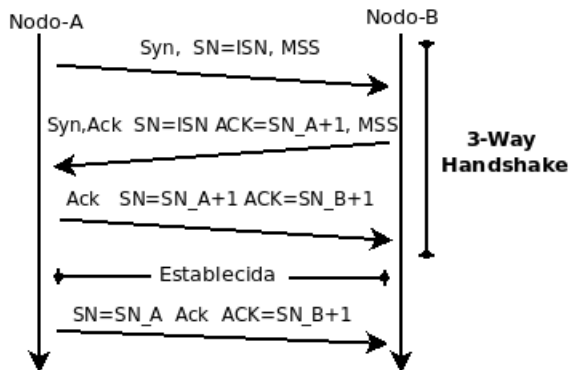
- **Source Port:** 16 bits. Indica el número de puerto de la aplicación en el nodo que envía el segmento.
- **Destination Port:** 16 bits. Indica el número de puerto de la aplicación que debe recibir el segmento.
- **Sequence Number:** 32 bits. Indica el número asignado al primer byte de datos contenidos en el segmento.
- **Acknowledgment Number:** 32 bits. Indica el siguiente número de byte que el emisor del segmento espera recibir del otro extremo.
- **Header Length:** 4 bits. Indica el número de palabras de 4 bytes de la cabecera TCP (entre 20 y 60 bytes).

TCP Header - Campos

- **Control Bits:** también conocidos como flags indica la finalidad del segmento.
- **Window Size:** 16 bits. la cantidad de bytes, a partir del número de reconocimiento en el segmento, que el emisor puede recibir.
- **Checksum:** 16 bits. Su cálculo es obligatorio y se hace sobre la todo el segmento más una pseudocabecera.
- **Urgent Pointer:** 16 bits. Válido si el flag U está seteado. Define el valor a sumar al número de secuencia que indica donde finalizan los datos urgentes.
- **Options:** hasta 40 bytes. Información opcional.

Establecimiento de la sesión

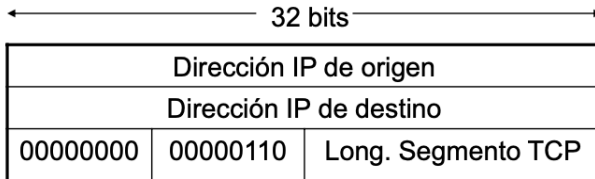
- Se intercambian 3 segmentos (3-way Handshake).
- También se indican los puertos origen y destino.
- Si en el destino no hay una aplicación escuchando en el puerto indicado, TCP responde con un mensaje con los flags R (Reset) y Ack setados. La sesión se cierra.



MSS

- Máxima longitud del segmento que se puede recibir. No cuenta la cabecera.
- Es una opción mandatoria (en cuanto a su implementación). No es un campo en la cabecera.
- Se envía solo en los mensajes con el flag S (SYN) del saludo de 3 vías.
- Si no se la envía se asumen los valores por default: 536 bytes en IPv4, 1220 en IPv6.
- Diferente para cada sentido de la conexión, aunque cada extremo debería tomar el mínimo informado.
- $MSS = MTU - (IPHeader + TCPHeader)$ (MTU=Maximu Transfer Unit)

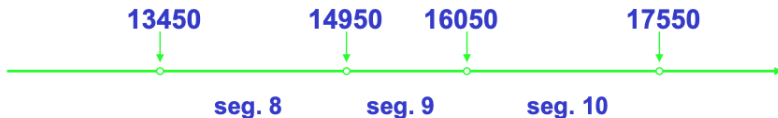
Pseudocabecera



- Se construye localmente solo para la generación/cálculo del checksum.
- No se envía.
- Verifica la integridad del segmento.
- El valor $110_2 = 6_{10}$ indica protocolo TCP.

Números de Secuencia

- Número de 32 bits.
- Valor inicial fijado en el establecimiento de la conexión a partir del ISN.
- ISN (Initial Sequence Number) determinado localmente en cada extremo.
- Cada byte de datos enviado sobre un conexión TCP tiene un número de secuencia.
- Se incrementa según los bytes transmitidos y reconocidos.
- Indica la ubicación de cada byte en el stream.



TCP - Intercambio de Datos

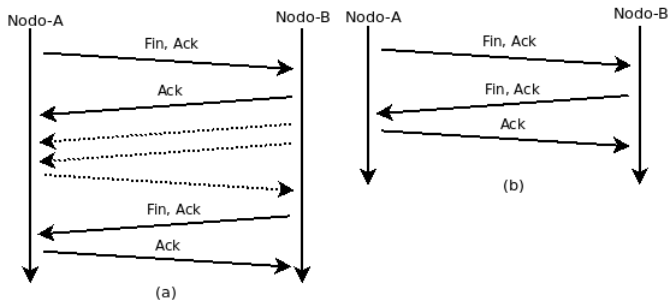
- Canal full-duplex o bidireccional.
- Un extremo puede enviar datos hasta el tamaño de la ventana de recepción recibido sin recibir un ACK.
- Si el segmento llega sin errores debe ser reconocido (ACK).
- **Piggybacking:** esperar a tener datos para enviar junto con el ACK.
- Número de ACK indica el siguiente byte que se espera recibir.
- ACKs son uniformemente incrementales y acumulativos.
- Si el segmento llega con errores, debe ser retransmitido.

TCP - Reset de Conexiones

- Un segmento con el flag R (Reset) seteado provoca un cierre inmediato de la conexión.
- Un extremo puede resetear una conexión en cualquier momento.
- TCP también puede enviar un reset ante determinados eventos.
- Un reset es enviado cuando se recibe un segmento que no se corresponda con la conexión referenciada.
- Si se recibe un intento de conexión a un puerto cerrado se debe responder con un mensaje con los flags R(Reset) y A(Ack).
- Si se produce un *half-open connection*, el extremo que cerró su conexión enviará un Reset.
- En el estado *SYN_SENT*, el reset se acepta si el valor del ACK reconoce el SYN. En el resto, el número de secuencia debe estar en la ventana del receptor.

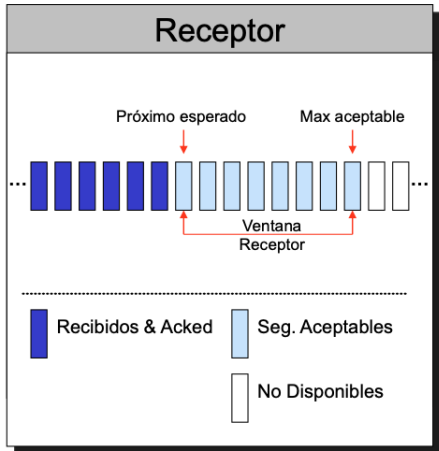
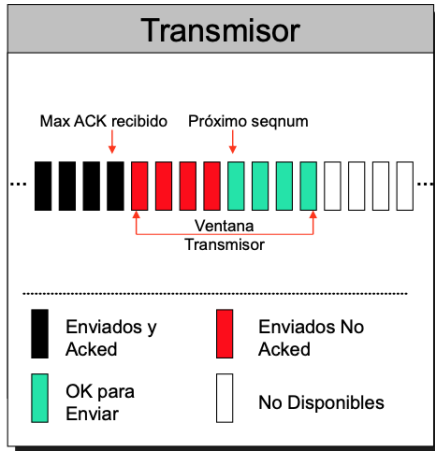
Fin de la sesión

- Se intercambian 3 (a) ó 4 (b) segmentos.
- Envío del flag F (Fin) no es el cierre definitivo de la sesión.
- Cualquiera de los extremos puede iniciar el cierre.
- En (a), Nodo B puede seguir enviando datos después de recibir el FIN de Nodo A, quien solo puede enviar ACKs.





Ventana deslizante



Control de Flujo

- Cada extremo anuncia el tamaño de su ventana de recepción en el inicio de sesión (pueden ser distintos).
- ACK de un segmento indica que fue recibido por TCP, no por la aplicación.
- Un extremo envía datos más rápido de lo que el otro extremo puede procesar.
- Si los datos no son leídos por la aplicación, el extremo debe anunciar su ventana de recepción con un menor tamaño
- Eventualmente, el tamaño de la ventana anunciada puede llegar a 0.

Ventana deslizante - Ejemplo

Segmento enviado

Source Port	Dest. Port
Sequence Number	
Acknowledgment	
HL/Flags	Window
D. Checksum	Urgent Pointer
Options..	

Segmento recibido

Source Port	Dest. Port
Sequence Number	
Acknowledgment	
HL/Flags	Window
D. Checksum	Urgent Pointer
Options..	



Retransmission Timeout(RTO) y RTT(Round Trip Time)

- RTO se debe establecer en base al RTT medido en una conexión determinada
- *RTT Sample*: tiempo desde que se envía un byte con un número de secuencia y vuelve el ACK que lo reconoce
- RTT varía continuamente:
 - Demasiado largo — > Subutilización
 - Demasiado corto — > Retransmisiones inútiles
- Desafío: determinar un buen estimado para el RTT y, en consecuencia, el RTO
- Solución:
 - Timeout dinámico, estimando el RTT

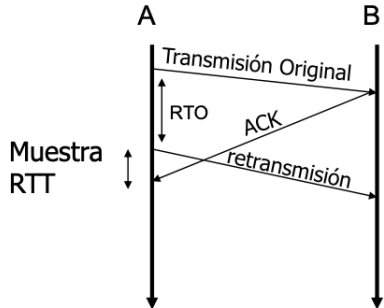
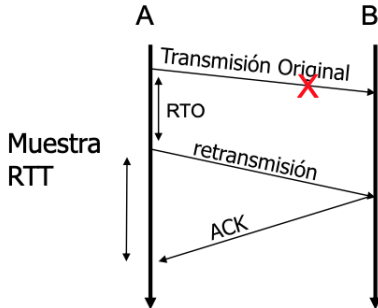
Estimando el RTT

- ✓ Teorema de Chebyshev's
- ✓ $\text{Max RTT} = \text{Avg RTT} + k \times \text{Desv}$
- ✓ $\text{Avg rtt} = (1 - y) \times \text{Avg RTT} + y \times \text{Muestra RTT}; y = 0,1$
- ✓ Probabilidad de error $\leq 1/(k^2)$
- ✓ Válida para toda distribución de muestras

$$\text{Timeout} = \text{Average RTT} + 4 * \text{Desv}$$

Se suele fijar en múltiplos de 200, 500, 1000 mseg

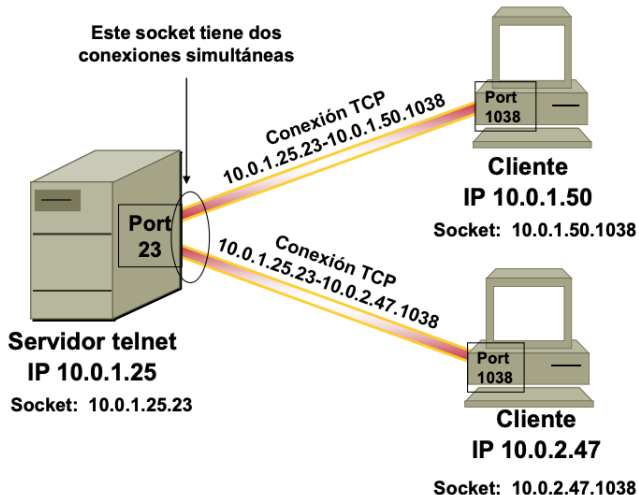
Ambigüedades



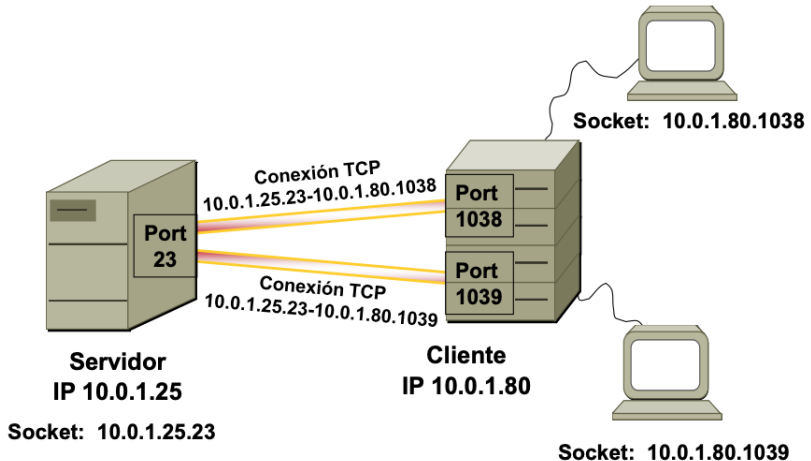
Estimador de Karn

- Ante una retransmisión:
 - No actualiza los RTT
 - Timer backoff = $RTO = 2 * RTO$
- Volver a estimar después de una transmisión exitosa
- Las muestras se toman en base a la opción de Timestamp de TCP

Sesiones a un mismo Servidor



Sesiones desde un mismo Host



Comando netstat/ss

U:\>netstat -a -n -p TCP

Active Connections

Proto	Local Address	Foreign Address	State
TCP	0.0.0.0:135	0.0.0.0:0	LISTENING
TCP	0.0.0.0:12345	0.0.0.0:0	LISTENING
TCP	10.10.2.38:139	0.0.0.0:0	LISTENING
TCP	10.10.2.38:1132	0.0.0.0:0	LISTENING
TCP	10.10.2.38:1132	10.10.6.170:139	ESTABLISHED
TCP	10.10.2.38:1200	10.10.6.99:1027	ESTABLISHED
TCP	10.10.2.38:1203	10.10.6.105:1306	ESTABLISHED
TCP	10.10.2.38:1207	0.0.0.0:0	LISTENING
TCP	10.10.2.38:1207	10.10.3.228:139	ESTABLISHED

Estructura Opciones TCP

- TCP puede tener hasta 40 bytes de opciones.
- Dos categorias de opciones: de un byte o multiple bytes.
- Todas las opciones comienzan con 1 byte, *kind*, que indica el tipo de opción.
- Campo Length para las opciones de multiple bytes.

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

NOP	...	Kind	Length	Data
Data				
Data			EOL	

Opciones

Table 13-1 The TCP option values. Up to 40 bytes are available to hold options.

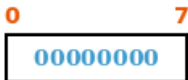
Kind	Length	Name	Reference	Description and Purpose
0	1	EOL	[RFC0793]	End of Option List
1	1	NOP	[RFC0793]	No Operation (used for padding)
2	4	MSS	[RFC0793]	Maximum Segment Size
3	3	WSOPT	[RFC1323]	Window Scaling Factor (left-shift amount on window)
4	2	SACK-Permitted	[RFC2018]	Sender supports SACK options
5	Var.	SACK	[RFC2018]	SACK block (out-of-order data received)
8	10	TSOPT	[RFC1323]	Timestamps option
28	4	UTO	[RFC5482]	User Timeout (abort after idle time)
29	Var.	TCP-AO	[RFC5925]	Authentication option (using various algorithms)
253	Var.	Experimental	[RFC4727]	Reserved for experimental use
254	Var.	Experimental	[RFC4727]	Reserved for experimental use

.-Extraído de TCP/IP Illustrated 2nd. edition

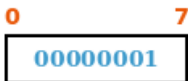
RFC 1323 fue actualizada por RFC 7323

Opciones...

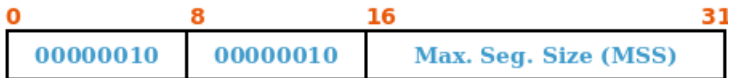
- **End of line:** 1 byte. Indica que no existen más opciones.
- Kind: 0



- **No-Operation (NOP):** 1 byte. Padding para que la siguiente opción inicie en el límite de 4 bytes.
- Kind: 1

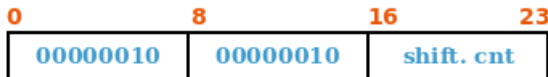


- **Maximum-Segment Size (MSS):** 4 bytes. Indica el tamaño de segmento más grande que espera recibir.
- Kind: 2, Length: 4.



Opciones ...

- TCP Window Scale: incrementa el tamaño de la ventana de recepción.
- Kind: 3, Length: 3.
- Enviado en el inicio de sesión. Indica el exponente del factor de escala.



- TCP TimeStamp: permite medir el round-trip time de cada paquete reconocido.
- Kind: 8, Length: 10.
- Se le anteponen dos NOPs para alineararlo a enteros de 32 bits.



TCP Persist Timer - RFC 1122

- Es necesario hacer una suerte de polling.
- Pueden producirse *deadlocks*.
- Al recibirse una ventana de 0 se activa el persist timer.
- Normalmente 5 segundos.
- Cuando expira se envía un segmento de 1 byte para verificar el estado del receptor.
- El receptor le contesta acorde con el estado en que se encuentra.
- Si el receptor vuelve a constestar con ACK , wind=0, entonces el persist timer hace un back-off binario (10, 20, 40 seg...).
- Al contestar el ACK no validando el byte recibido el emisor continúa enviando este byte de prueba.
- La diferencia con el RTO es que en este caso el emisor envía permanentemente esta prueba hasta que la ventana se incremente o se haga un reset de la conexión.

Keepalive Timer - RFC 1122

- En TCP si no hay intercambio de datos no hay tráfico alguno, pero la conexión persiste.
- Hasta que haya una caída en algún extremo o reboot de alguno de los hosts.
- Se define para interrogar al otro extremo por su estado.
- No es parte de la especificación de TCP.
- No se recomienda porque puede provocar caídas en caso que la falla sea transitoria.
- También incrementa el uso del ancho de banda disponible como cualquier otra acción de control
- En algunos casos es necesario. Ej., Telnet.

Keepalive Timer... I

- En el extremo en que esté habilitado si no hay actividad en un lapso de tiempo configurable envía un segmento de prueba.
- El segmento de prueba es similar al de persist timer.
- Puede también enviarse vacío como un ACK pero con un número de secuencia inesperado.
- El receptor se puede encontrar en alguno de estos estados:
 - Activo:** Responderá al segmento de prueba. El emisor resetea el timer por otras 2 horas. Si en ese intervalo aparece tráfico entonces se vuelve a resetear.
 - Caído:** o en proceso de reboot. El emisor no recibirá respuesta y se genera un timeout a los 75 segundos. Repite 10 veces en intervalos de 75 seg. Si no recibe respuesta finaliza la conexión
 - Fin-reboot:** el emisor recibirá una respuesta que será un reset de la conexión.

Keepalive Timer... II

Receptor activo pero inalcanzable por el emisor: el emisor no recibe respuesta y genera un timeout de 75 seg al cabo de los cuales retransmite el byte de prueba. Es similar al caso 2.

Wrap around

- El número de secuencia tiene 32 bits
- En velocidades altas el espacio de 32 bits puede reciclar dentro del intervalo de tiempo que un segmento es retardado en la red
- Para conseguir una operación libre de este error:

$$2^{31} / B > MSL(seg)$$

donde B es la velocidad efectiva del enlace en bytes/seg

$$T_{wrap} = 2^{31} / B$$

Red	B (Mbps)	Twrap (seg)
Ethernet	1.25	1700
FDDI	12.5	170
Gigabit	125	17



PAWS - RFC 1123-7323

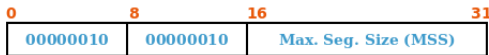
- Definido para rechazar segmentos duplicados y reencarnaciones
- Utiliza la opción de timestamp
- Asume que los segmentos de datos y ACK recibidos contienen un valor de TS monótono no-decreciente
- Un segmento puede ser descartado como duplicado si se recibe con un valor de TS menor que uno anterior
- *Menor que* significa que si s y t son valores de TS, entonces

$$s < t \text{ si } 0 < (t - s) < 2^{31}$$

- Los valores de TS enviados en $\langle \text{SYN} \rangle$ y/o $\langle \text{SYN}, \text{ACK} \rangle$ inicializan PAWS
- No requiere sincronización de relojes entre emisor y receptor

SACK - RFC 2018

- Permite la retransmisión selectiva.
- RFC 2018.
- Comprende dos opciones de TCP:
 - “Sack-Permitted Option”
 - “Sack Option”
- **Sack Permitted Option:**
 - Enviada en el segmento SYN del saludo de 3 vías.
 - Habilita el uso de SACK una vez que la conexión esté establecida.
 - Kind: 4, Length: 2.



Sack Option – Estructura

- NOP se agregan para alinear la opción a palabras de 32 bits (4 bytes).
- Cada bloque indica un rango de números de secuencias que el receptor ha recibido correctamente.
- No se pueden indicar más de 4 huecos.
- Left Edge of Block: indica el primer número de secuencia del bloque.
- Right Edge of Block: indica el número de secuencia inmediato siguiente al último número de secuencia del bloque.

0	7	15	23	31
NOP	NOP	00000101	Length	
Left Edge of 1st. Block				
Right Edge of 2nd. Block				
...				
Left Edge of nth. Block				
Right Edge of nth. Block				

Ejemplos SACK

El transmisor envía 8 segmentos de 500 bytes de datos con SN:5000

Caso 1:

Se pierde el primer segmento y llegan OK los 7 restantes:

Triggering Segment	ACK	Left Edge	Right Edge
5000	(lost)		
5500	5000	5500	6000
6000	5000	5500	6500
6500	5000	5500	7000
7000	5000	5500	7500
7500	5000	5500	8000
8000	5000	5500	8500
8500	5000	5500	9000

Ejemplos SACK...

El transmisor envía 8 segmentos de 500 bytes de datos con SN:5000

Caso 2:

Se pierden los segmentos pares:

Triggering Segment	ACK	1er Bloque		2do Bloque		3er Bloque	
		Left Edge	Right Edge	Left Edge	Right Edge	Left Edge	Right Edge
5000	5500						
5500	(lost)						
6000	5500	6000	6500				
6500	(lost)						
7000	5500	7000	7500	6000	6500		
7500	(lost)						
8000	5500	8000	8500	7000	7500	6000	6500
8500	(lost)						

Ejemplos SACK...

Continuando con el caso anterior, supongamos que se recibe el cuarto paquete a continuación. Entonces:

Triggering Segment	ACK	1er Bloque		2do Bloque		3er Bloque	
		Left	Right	Left	Right	Left	Right
		Edge	Edge	Edge	Edge	Edge	Edge
6500	5500	6000	7500	8000	8500		

Si luego llega el segundo segmento:

Triggering Segment	ACK	1er Bloque		2do Bloque		3er Bloque	
		Left	Right	Left	Right	Left	Right
		Edge	Edge	Edge	Edge	Edge	Edge
5500	7500	8000	8500				



**Atribución-NoComercial-CompartirIgual
4.0 Internacional (CC BY-NC-SA 4.0)**

Esta obra está sujeta a la licencia Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0) de Creative Commons.

Para detalle de esta licencia visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/>